# VDX Studio Turbo — Project Documentation

---

## 1. Core Features

### Creative Authoring & Editing

- **Simple Edit** — streamlined creative editing workflow (`/simple-edit` route, `features/simple-edit`)
- **Advanced Edit** — full-featured creative editor packaged as a shared workspace library (`@turbo/advanced-edit`, `/advanced-edit` route)
- **Sprout authoring** — dedicated Sprout creative format with its own editor, revisions view, and preview (`/sprout`, `/sprout/editor`)
- **Sprout Scripts** — script-driven creative generation with vertical/brand/product/script views, script editor, and script preview pages
- **Rich-text editing** — Tiptap-based text editing (headings, lists, links, images, colors, font families) embedded in the UI library
- **Code editing** — in-browser HTML/CSS editing via CodeMirror 6 and Monaco editors (linting, autocomplete, search, one-dark theme)
- **Rich media creatives** — dedicated `richmedia` API module and controller for rich media ad units

### Project & Asset Management

- **Projects and revisions** — CRUD, revision history, and per-project screenshots (`projects`, `revisions`, `project-screenshots` controllers)
- **Templates** — template library with template requests and user-specific templates (`templates`, `template-requests`, `user-templates` controllers)
- **Asset management** — upload, storage, and serving of creative assets (`assets` module, archiver/zip handling, image sizing via `sharp` and `image-size`)
- **PSD import pipeline** — Adobe Photoshop UXP plugin (`apps/psd`) that extracts layers/fonts (opentype.js, pako, jszip) and a matching `psd` API module for template conversion
- **Tagging** — tag creation and assignment across entities (`tags` module)
- **Favorites & likes** — per-user favorites and likes on creatives (`favorites`, `likes` modules)
- **Brand management** — brand records and brand-scoped workflows (`brands` module)
- **Home dashboard** — recent items, projects, and navigation hub (`/home`, `dashboard` feature)

### Review & Collaboration

- **Client Showcase** — separate client-facing app (`apps/client-showcase`) for previewing creatives, with demo pages, ad-size selection, feature selection, and settings
- **Review & annotation** — review workflows with commenting/mentions (Tiptap mention extension) and on-creative markup via marker.js (`review` module, `review` feature)
- **Video preview with ad simulation** — video.js with IMA/ads plugins (`videojs-ima`, `videojs-contrib-ads`) for in-context ad playback
- **QR code hand-off** — QR generation (`qr-code-styling`, `qrcode.react`) and scanning (`qr-scanner`) for previewing creatives on devices
- **Real-time updates** — WebSocket gateway on the API (`events.gateway.ts`) and `react-use-websocket` in the showcase for live sync
- **Email notifications** — SMTP mail via Nodemailer with Mailpit for local capture; sprout email sending (`sprout-emails` controller)

### Media & Rendering Services

- **Video/media processing** — dedicated NestJS `ffmpeg` microservice using fluent-ffmpeg for transcoding and media operations
- **Automated screenshots** — external screenshots service (Dockerized) plus `screenshots` API module for capturing creative previews
- **Text-to-speech / voice** — ElevenLabs integration (`@elevenlabs/elevenlabs-js`, `elevenlabs` module)
- **Geo/location features** — Mapbox maps and geocoding, POI (points of interest) module with locations controller, geo-token service integration

### Platform & Operations

- **Authentication & authorization** — JWT-based auth (RS256 key pair via Docker secrets), Passport local + JWT strategies, protected routes in the web app
- **User management** — user accounts and profiles (`users` module)
- **Background jobs** — queued/asynchronous task processing via BullMQ (`tasks`, `expotask` modules)
- **Analytics & usage tracking** — `analytics` module plus BigQuery-backed sprout usage facts
- **Incident alerting** — PagerDuty integration (`pd` module)
- **External/partner API** — `external` module and controller for third-party access
- **Maintenance & support** — `support` and `maintenance` controllers for operational endpoints
- **Structured logging** — Winston with daily-rotate-file, shared `logs` package, request-context logging via `nestjs-cls`
- **End-to-end testing** — Playwright test suite with snapshot testing (`playwright/` workspace, dev and prod targets)

---

## 2. Frontend Stack

### Core Frameworks & Build

- **React 18** — UI framework for `web` and `client-showcase` apps
- **TypeScript 5.9** — static typing across all frontend code

- **Vite 7** — dev server and bundler (with `@vitejs/plugin-react-swc` / SWC for fast transforms)
- **Webpack 5 + Babel** — build toolchain for the Photoshop UXP plugin (`apps/psd`)
- **React Router DOM v6** — client-side routing (`createBrowserRouter`)

### State Management & Data Fetching

- **Recoil** — global client state in the web app
- **TanStack React Query v5** — server-state fetching/caching (with devtools)
- **Axios** — HTTP client
- **Zod v4** — runtime schema validation
- **localforage** — client-side persistence
- **jwt-decode** — token parsing on the client

### UI, Styling & Interaction

- **Tailwind CSS 3** — utility-first styling (with `@tailwindcss/container-queries`, PostCSS, Autoprefixer)
- **Sass** — additional styling in the showcase app
- **Framer Motion** — animation
- **@turbo/ui** — internal shared component library
- **@turbo/icons** — internal shared icon library
- **@ebay/nice-modal-react** — declarative modal management
- **react-resizable-panels** — split-pane editor layouts
- **@tanstack/react-table + react-virtual** — data grids and list virtualization
- **react-infinite-scroll-component** — infinite scrolling lists
- **react-day-picker** — date selection
- **classnames** — conditional class composition
- **usehooks-ts** — utility React hooks

### Editors & Rich Content

- **Tiptap 2** — rich-text editing (starter kit, mentions, text-style, blockquote, and many extensions)
- **CodeMirror 6** — HTML/CSS code editing with linting, search, and autocomplete
- **Monaco Editor** (`@monaco-editor/react`) — secondary code editing surface
- **js-beautify** — code formatting

### Visualization & Media

- **amCharts 5** — charts and analytics visualization
- **Mapbox GL + Geocoder + Turf.js** — interactive maps, address search, geo-circle calculations
- **video.js + videojs-ima + videojs-contrib-ads** — video playback with ad insertion
- **html-to-image** — DOM-to-image capture
- **qr-code-styling / qrcode.react / qr-scanner** — QR code generation and scanning
- **react-frame-component** — iframe-isolated creative rendering
- **jszip** — client-side zip packaging
- **jsoncrack-react** — JSON structure visualization

---

## 3. Backend Stack

### Languages & Frameworks

- **Node.js 24** — runtime (Alpine-based Docker images)
- **TypeScript 5.9** — all backend source
- **NestJS 10** — primary backend framework for both the `api` and `ffmpeg` services (modules, DI, guards, pipes)
- **Express** — HTTP layer under NestJS (`@nestjs/platform-express`) and standalone in `client-showcase-api`
- **tsx** — TypeScript runtime for the lightweight `client-showcase-api` service

### Core API Dependencies

- **Kysely** — type-safe SQL query builder for PostgreSQL (`pg` driver), with entities, repositories, and migrations
- **Passport + @nestjs/jwt** — authentication (local and JWT strategies, RS256 keys)
- **BullMQ + @nestjs/bullmq** — Redis-backed job queues
- **KafkaJS (+ snappy codec)** — Kafka event production/consumption
- **@nestjs/websockets / platform-ws + ws** — WebSocket gateway for real-time events
- **@nestjs/swagger** — OpenAPI spec generation (consumed by `openapi-typescript` for frontend types)
- **class-validator + class-transformer + Joi** — request validation and config validation
- **nestjs-cls** — request-scoped context (params, cookies, tab id) for logging/tracing
- **@nestjs/event-emitter** — in-process domain events
- **@nestjs/cache-manager** — application-level caching
- **Winston + winston-daily-rotate-file** — structured, rotated logging
- **Nodemailer** — SMTP email delivery
- **sharp / image-size / file-type** — image processing and file introspection

- **archiver / node-stream-zip** — zip creation and reading
- **fluent-ffmpeg** — media processing (ffmpeg service)
- **opentype.js** — font parsing (PSD/text pipeline)
- **@google-cloud/bigquery** — analytics warehouse client
- **@elevenlabs/elevenlabs-js** — text-to-speech API client
- **express-http-proxy + nanoid** — proxying and id generation in `client-showcase-api`

### Tooling & Quality

- **Turborepo** — task orchestration and caching across the monorepo
- **npm workspaces (npm@10)** — package management (`apps/*`, `packages/*`, `playwright/*`)
- **Biome** — linting/formatting for app code; **ESLint + Prettier** for shared configs
- **Husky + lint-staged** — pre-commit format/lint hooks
- **patch-package** — dependency patching (`patches/`)
- **Jest 30 + ts-jest + Supertest** — unit and e2e API testing
- **Playwright** — browser end-to-end tests with snapshots

---

## 4. Architecture & Patterns

### Repository & Code Organization

1. **Turborepo monorepo** — apps (`api`, `web`, `client-showcase`, `client-showcase-api`, `ffmpeg`, `psd`) plus shared packages, with dependency-aware `build` pipelines (`dependsOn: ^build`) and task caching
2. **Layered shared-package architecture** — `document-module` (core document model) `adapters api-interfaces` / `vdx-document sync-module @turbo/ui` / `@turbo/advanced-edit`, consumed by both frontend and backend
3. **Shared type contracts** — `api-interfaces` and `showcase-types` packages keep client and server types in sync; Swagger JSON is additionally converted to frontend types via `openapi-typescript` (`generate-api-types`)
4. **Feature-based frontend structure** — `features/`, `pages/`, `components/`, `services/`, `state/`, `hooks/` separation in both React apps

### Backend Design Patterns

1. **Modular monolith (NestJS)** — ~30 feature modules (projects, templates, assets, sprouts, review, brands, analytics, etc.) with controller  service repository layering
2. **Microservice split for heavy work** — separate `ffmpeg` service (media) and external screenshots service, called over HTTP from the API
3. **Repository pattern + migrations** — database access isolated in `database/entities`, `database/repositories`, and versioned `database/migrations` (Kysely)
4. **Dependency injection & config factories** — typed config providers (`DatabaseConfig`, `JwtConfig`, `KafkaConfig`, etc.) loaded through `@nestjs/config` with `forRootAsync` module wiring
5. **Guard/strategy-based auth** — Passport strategies with JWT public/private keys injected as Docker secrets; protected-route wrapper on the frontend
6. **Request-context propagation** — CLS middleware captures request metadata (URL, method, cookies, tab id) for cross-cutting logging
7. **Global exception handling** — `all-exceptions.filter.ts` centralizes error responses

### Data Flow & Communication

1. **REST API** — resource-oriented controllers documented with Swagger (web/showcase clients use Axios + React Query)
2. **WebSockets** — API `EventsGateway` pushes real-time events; showcase uses `react-use-websocket`
3. **Event-driven messaging** — Kafka (KRaft mode) used for asynchronous events (e.g., screenshots workflow); in-process events via EventEmitter
4. **Queued background jobs** — BullMQ on Redis for deferred/async task processing
5. **Proxy pattern** — `client-showcase-api` proxies and mediates showcase traffic; Nginx templates for proxy/web-prod/showcase-prod fronting in production images

### Development & Delivery Patterns

1. **Docker-first local environment** — single `docker compose up --watch` stack with hot-reload: source volumes for app code and Compose `develop.watch` sync+restart rules for shared package `dist` outputs
2. **Separate dev vs. prod Dockerfiles** — `*-dev.Dockerfile` for watch mode, multi-stage `*-prod.Dockerfile` images (web served by Nginx, API by Node), plus stack files for dev environments (`docker-stack.dev*.yml`)
3. **Secrets as files** — JWT keys and BigQuery service account mounted via Docker secrets, not env vars
4. **Remote debugging support** — Node inspector port (9229) exposed with a documented VS Code attach configuration
5. **Trunk-based flow with PR reviews** — `main` branch with squash-merged pull requests tied to Jira-style ticket ids (VDXST-xxxx)

---

## 5. Infrastructure & Data

### Data Stores

- **PostgreSQL 16 (Alpine)** — primary relational database (`vdxstudio` DB, accessed via Kysely; separate read-only credentials supported)
- **Redis 7.4** — cache and BullMQ job-queue backend

- **Apache Kafka 3.6 (Bitnami, KRaft mode)** — event streaming broker (ports 9092/9093)
- **Google BigQuery** — analytics warehouse (`facts.sprout_usage` table, service-account auth)
- **File-system storage** — mounted public/upload directories (`/mnt/vdxstudioturbo`, media dirs) for assets, projects, and screenshots; NetApp-backed paths in shared environments
- **Docker named volumes** — `pgdata` for Postgres persistence

### Local/Platform Services (Docker Compose)

- **web** — Vite dev server behind port 8080
- **api** — NestJS API on port 3000 (plus 9229 debug, 8081)
- **ffmpeg** — media service on port 3001 (amd64 image)
- **screenshots** — company-registry image (`vdx-builder/screenshots`) on port 8070
- **mail (Mailpit)** — local SMTP capture with web UI on 8025
- **postgres / redis / kafka** — backing stores listed above
- **Nginx** — reverse proxy and static hosting in production images (proxy, web-prod, showcase-prod templates)

### External APIs & Integrations

- **Expo9 / Exponential API** (`api.exponential.com`) — core ad-platform integration
- **ElevenLabs** — text-to-speech generation
- **Mapbox** (`api.mapbox.com`) — maps, geocoding, and location search
- **PagerDuty** — incident/event alerting
- **AE Template Builder (AETB)** — After Effects dynamic template service
- **VRS** — video rendering service integration
- **POI service** — points-of-interest data provider
- **Token / Geo-token service** — token issuance for geo features
- **Dotter service** — internal creative service integration
- **CS (Content/Creative Service)** — internal service consumed via `cs` module (VRS controller included)
- **CDN / tag hosting** (`tribalfusion.com` domains) — creative tag delivery and QR-code landing (`e9tag.html`)
- **Company Docker registry** (`expodockerregistry.tf-net.tribalfusion.com`) — private base images (ffmpeg, screenshots)
- **Stash (Bitbucket Server)** (`stash.exponential.com`) — Git hosting over SSH, behind FortiClient VPN

### Environments & Access

- **Local** — full Docker Compose stack (`npm run docker:dev`), web at `http://localhost:8080`
- **Dev/staging stacks** — Swarm-style stack files (`docker-stack.yml`, `docker-stack.dev73/300/806.yml`) targeting internal dev hosts
- **Staging CDN** — `beta-staging.tribalfusion.com` for previews and QR tags
- **VPN-gated infrastructure** — internal registries, Stash, and service endpoints require the company VPN

---

# VDX Studio Turbo — Supported Web Components

> *Inventory of the React components supported across the Turbo Project, generated from codebase analysis on 2026-07-30. Components are grouped by the package or app that owns them.*

---

## 1. Shared UI Library — `@turbo/ui` (`packages/ui`)

> *The core reusable component library, consumed by `web`, `client-showcase`, and `@turbo/advanced-edit`. Built with tsup, styled with Tailwind CSS.*

### 1.1 Exported UI Components (`src/components`)

- **AdvancedSearch** (`advanced-search`) — advanced/filtered search control
- **CodeEditor** (`code-editor`) — CodeMirror 6-based HTML/CSS code editor with code controls, custom extensions, a Tiptap variant, and a `useCodeEditor` hook
- **ColorPicker** (`color-picker`) — color selection control
- **CustomAvatar** (`custom-avatar`) — user avatar display
- **ImageCarousal** (`image-carousal`) — image carousel/slideshow
- **ImagePreview** (`image-preview`) — image preview surface
- **Origin** (`origin`) — origin/anchor-point selector (element positioning)
- **Slider** (`slider`) — range slider input

- **Timeline** (`timeline`) — timeline view with `Timeline` and `TimelineItem` components
- **TypeaheadExt** (`typeahead-ext`) — typeahead/autocomplete input

## 1.2 Dynamic Properties System (`src/dynamic-properties`)

> *Schema-driven property panel used by the editors to render creative-element properties.*

- **PropertyView** (`property-view.tsx`) — top-level renderer for a property panel

### Property Widgets (`widgets/`)

- **AdMenu** (`admenu`) — ad menu configuration widget
- **AdTab** (`adtab`) — ad tab configuration widget
- **Arrows** (`arrows`) — arrow/navigation controls widget
- **Divider** (`divider`) — divider element widget
- **Fill** (`fill`) — fill/background styling widget
- **Font** (`font`) — typography/font settings widget
- **Frame** (`frame`) — frame/layout settings widget
- **Image** (`image`) — image source and settings widget
- **MissingWidget** (`missing-widget.tsx`) — fallback for unknown widget types

### Property Controls (`components/`)

- **ColorControl** — color value input
- **InputControl** — generic text/number input
- **InputWithMixedIndicator** — input that flags mixed multi-selection values
- **LinkedInputControl** — linked value inputs (e.g., proportional width/height)
- **SwitchControl** — boolean toggle switch
- **ToggleIconControl** — icon-based toggle
- **WidgetContainer / WidgetFeature / WidgetLayout** — structural wrappers for widget composition
- **DebugComponentSelect** — development-time widget selector

## 1.3 Editor Wrapper (`src/editor-wrapper`)

> *The document-editing runtime that binds the UI to the shared document model.*

### Components & Contexts

- **TurboEditor** (`turbo-editor.tsx`) — main embeddable creative editor component
- **TurboDocumentContext** — document-level React context provider
- **TurboEditorContext** — editor-level React context provider

### Editor Hooks (supported API)

- `useTurboDocument` / `useTurboEditor` — access the active document/editor
- `useNodeProperty` — read/write a node's property
- `useSelected` / `useFocused` — selection and focus state
- `usePayloadValue` / `useMutatePayloadValue` — document payload access and mutation
- `useKeyValue` / `useLocalState` — key-value and local editor state
- `useUiBinding` — bind UI controls to document values
- `useExtension` / `useExtensionClass` — editor extension access
- `useCss` — CSS handling for the edited creative
- `useExecWithSync` — execute mutations with sync coordination
- `useEditorReload` — editor reload lifecycle
- `useOnDocumentReadyChange` — document-ready lifecycle callback
- `useNodeIdsByTag` — query nodes by tag
- `useNodeDebug` — node debugging support
- `MixedValue` (`mixed-value.ts`) — mixed multi-select value handling

---

# 2. Icon Library — `@turbo/icons` (`packages/icons`)

- **210 individual SVG icon components**, one React component per file
- Categories include (examples):
  - **Alignment** — `AlignHorizontalLeft/Center/Right`, `AlignVerticalTop/Center/Bottom`
  - **Creative/ad concepts** — `AdSize`, `Adhesion`, `CTALink`, `CTVOTT`, `CcrCcrv`, `Callout`, `AllTemplates`
  - **Effects** — `Blur`, `Bokeh`
  - **Brand/product logos** — `ATPLogo`, `ApolloLogo`, `BoxFileStorageLogo`

- **General UI** — `ArrowUpRight`, `At`, `Bookmark`, `Bulb`/`BulbActive`, `Calendar`, `ChartBar`, `Chat`, `Check`, `CheckCircle`, `CheckCircle2`, and ~180 more

---

## 3. Advanced Editor Package — `@turbo/advanced-edit` (`packages/advanced-edit`)

> *The advanced-editing workspace mounted at `/advanced-edit` in the web app.*

- **AdvancedCanvas** (`advanced-canvas`) — main WYSIWYG canvas for creative editing
- **Canvas tools** (`canvas-tool`):
  - **BaseCanvasTool** — shared tool infrastructure
  - **SelectorTool** — element selection/transform tool
  - **TextTool** — on-canvas text editing tool
- **Navigation** (`navigation`):
  - **PageGroup / PageGroupSelector** — page-group navigation
  - **PageSelector** — page switching
  - **TreeExplorer** — layers/document tree explorer
- **PropertyEditor** (`property-editor`) — property panel integration for the advanced canvas

---

## 4. Web App Components — `apps/web/src/components`

> *App-level components specific to the main studio UI.*

- **AppSidebar** (`app-sidebar`) — main application navigation sidebar
- **AssetsUpload** (`assets-upload`) — asset upload UI
- **AuthContext** (`auth-context`) — authentication context provider
- **CtvOttCanvas** (`ctv-ott-canvas`) — CTV/OTT creative canvas preview
- **DynamicImage** (`dynamic-image`) — dynamic image rendering
- **EditListValues** (`edit-list-values`) — list-value editing control
- **ErrorBoundary** (`error-boundary.tsx`) — React error boundary
- **Header** (`header`) — top application header
- **Image** (`image.tsx`) — image display wrapper
- **Jira** (`jira`) — Jira integration UI (ticket linking/reporting)
- **LayersSearchInput** (`layers-search-input`) — search within document layers
- **Layout** (`layout`) — page layout scaffolding (plus `app-layout.tsx` / `app-layout-simple.tsx` pages wrappers)
- **Maintenance** (`maintenance`) — maintenance-mode screen
- **NodeInspector** (`node-inspector`) — document node inspection/debugging panel
- **PropertyRecord** (`property-record.tsx`) — property record display
- **ProtectedRoute** (`protected-route`) — auth-guarded route wrapper
- **SimpleCanvas** (`simple-canvas`) — canvas used by the Simple Edit workflow
- **SortingOptions** (`sorting-options.tsx`) — list sorting control
- **SuffixInput** (`suffix-input.tsx`) — input with unit/suffix display
- **Table** (`table`) — data table components (TanStack Table-based)
- **TasksStatus** (`tasks-status`) — background task status indicator
- **ZoomControls** (`zoom-controls`) — canvas zoom controls

---

## 5. Client Showcase Components — `apps/client-showcase/src/components`

> *Components for the client-facing preview/review application.*

- **AttachementViewModal** — attachment viewing modal
- **ColorPicker** and **custom-color-picker** — color selection controls
- **ConfirmationModal** — confirm-action dialog
- **Editor / EditorFooter** — showcase-side editing surface and footer actions
- **Error / PageError** — inline and page-level error displays
- **IconButtonWithTooltip** — icon button with tooltip
- **InformationModal** — informational dialog
- **MobileBreadcrums** — mobile breadcrumb navigation
- **ScrollButton** — scroll-to-position button
- **SearchInput** — search field
- **Layout** (`layout`) — showcase page layout components
- **Tiptap suite** (`tiptap/`):
  - **TiptapEditor** — rich-text editor for comments/review
  - **TiptapControlsBar** — formatting toolbar
  - **MentionControl / MentionList** — @-mention support
  - **LinkModal** — hyperlink insertion dialog

- **Extensions** — configured Tiptap extension set

---

## 6. Notes & Conventions

- **Component ownership** — reusable, cross-app components live in `packages/` (`@turbo/ui`, `@turbo/icons`, `@turbo/advanced-edit`); app-specific components stay in each app's `src/components`
- **Export pattern** — packages use barrel files (`index.ts`) re-exporting each component folder; anything exported from `packages/ui/src/index.ts` is the supported public surface
- **Styling** — Tailwind CSS utility classes with co-located `*.styles.ts` files; showcase additionally uses Sass
- **Editing stack** — rich text is Tiptap-based; code editing is CodeMirror 6-based (Monaco available in the web app)
- **Hot reload** — `packages/ui` and `packages/advanced-edit` sources are watched by Docker Compose (`develop.watch`) so component changes sync into the running web container